

RBE 1001

Final Project

**Fruit Harvesting Robot
Engineering Journal &
Documentation**

Table of Contents

Title Card	3
Problem Analysis	4
Initial Concept	7
Design Philosophy	8
Holonomic Drivetrain	9
Manipulator	10
Electronics / Sensor Array	11
Programming - State Structure	12
Programming - Navigational Methods	15

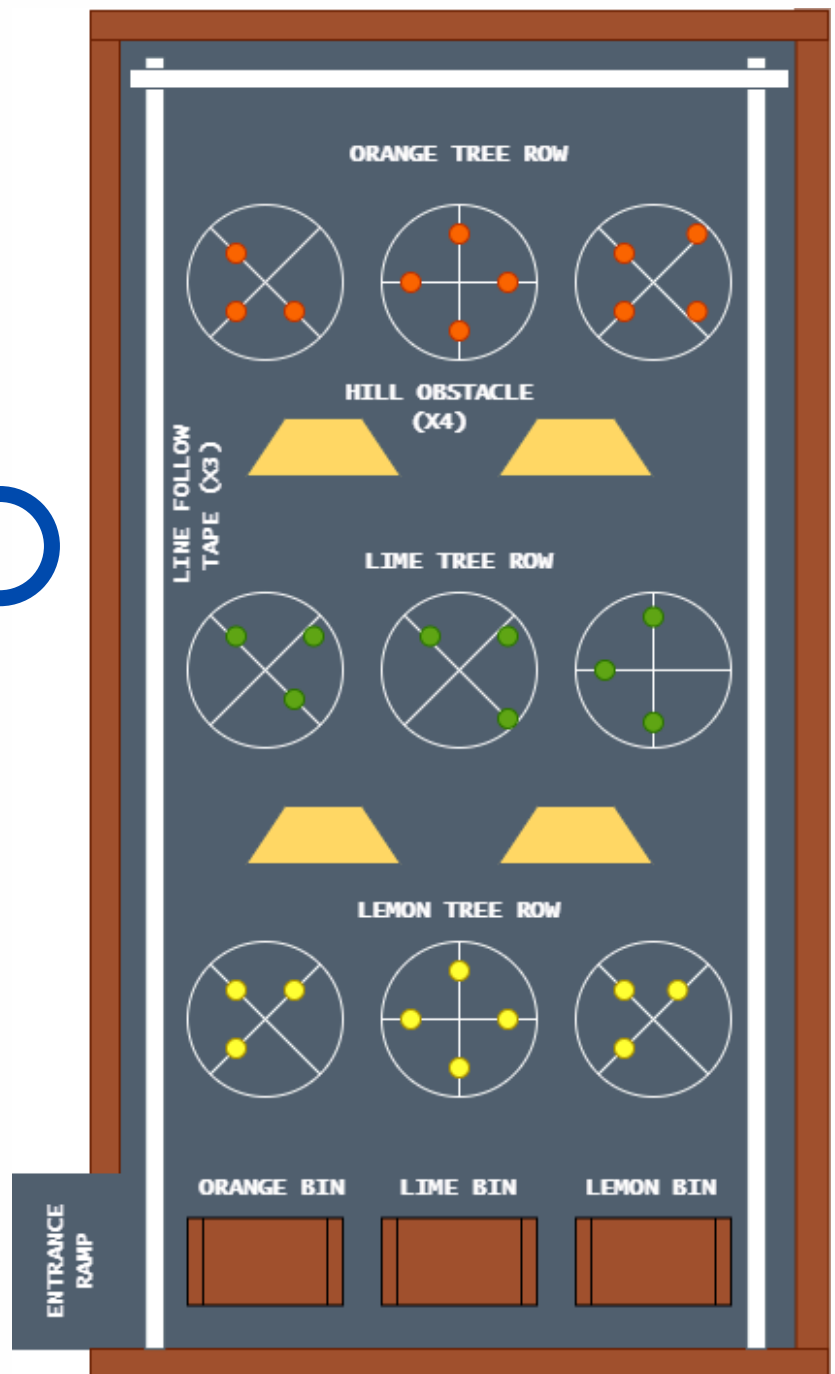
HighWater



Problem Analysis

HighWater is a robot created to complete a fruit-harvesting simulation involving autonomous navigation and harvesting of 3d-printed fruit props located on PVC trees. One of the main goals of the project was to familiarize students with good goal setting habits, and as such groups were allowed to decide what the capabilities of their final robot would be, with grades being based on how well the final design met the proposed set of end goals.

Field Map



Problem Analysis

Goal Setting

Working on this project gave all of our group members a better understanding of why proper goal setting is so difficult. To begin the process, we first identified what our goals were for what we wanted to get out of the project. The main points we settled on were:

- Demonstrate an ability to write concise, well structured state machine code
- Gain experience with vision processing & proportional control loops
- Demonstrate an ability to work within given design constraints
- Demonstrate an ability to create a fully autonomous machine guided by sensor data
- Gain experience in 3D modeling components

After generating this general list of our collective goals for the final project, we began to create our list of specific *needs*. These needs represent the things our system must be able to do to fully accomplish our goals as students.

Problem Analysis

System Needs

1. Need:

The system must be able to autonomously traverse the field

2. Need:

The system needs to autonomously identify fruit and align it's manipulator with fruit for collection

3. Need:

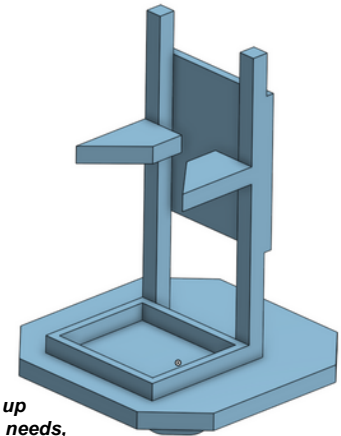
The system must either fit inside of the transportation box OR drive up the ramp

4. Need:

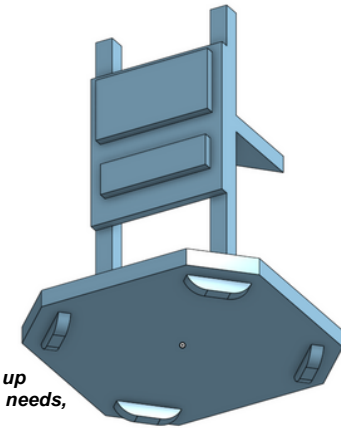
The system needs to be capable of both storing picked fruits and expelling them from the robot

5. Need:

The system must be able to detach fruit from the tree



Initial CAD mock up based on system needs, Front



Initial CAD mock up based on system needs, Back

Requirement Analysis

From our generated list of system needs, we were able to generate a list of system *requirements*. These requirements essentially represent a further breakdown of what exactly the system must be able to do to accomplish a need. This method of breaking down a task into needs and requirements of those needs was greatly beneficial in the design process, and can be repeated as many times as needed until the entire overall project is distilled into a series of small, achievable goals.

The project demanded a set of requirements for each of the above 5 primary needs. As an example of these requirements, here were the requirements for the first need, “The system must be able to autonomously traverse the field” :

Requirement: Drivetrain

The drivetrain must be capable of consistent and reliable maneuvering throughout the entire orchard terrain, excluding exceedingly rough terrain or obstacles.

Requirement: Navigation

The system must be able to reliably navigate to set positions on the field, using sensory data to inform decisions.

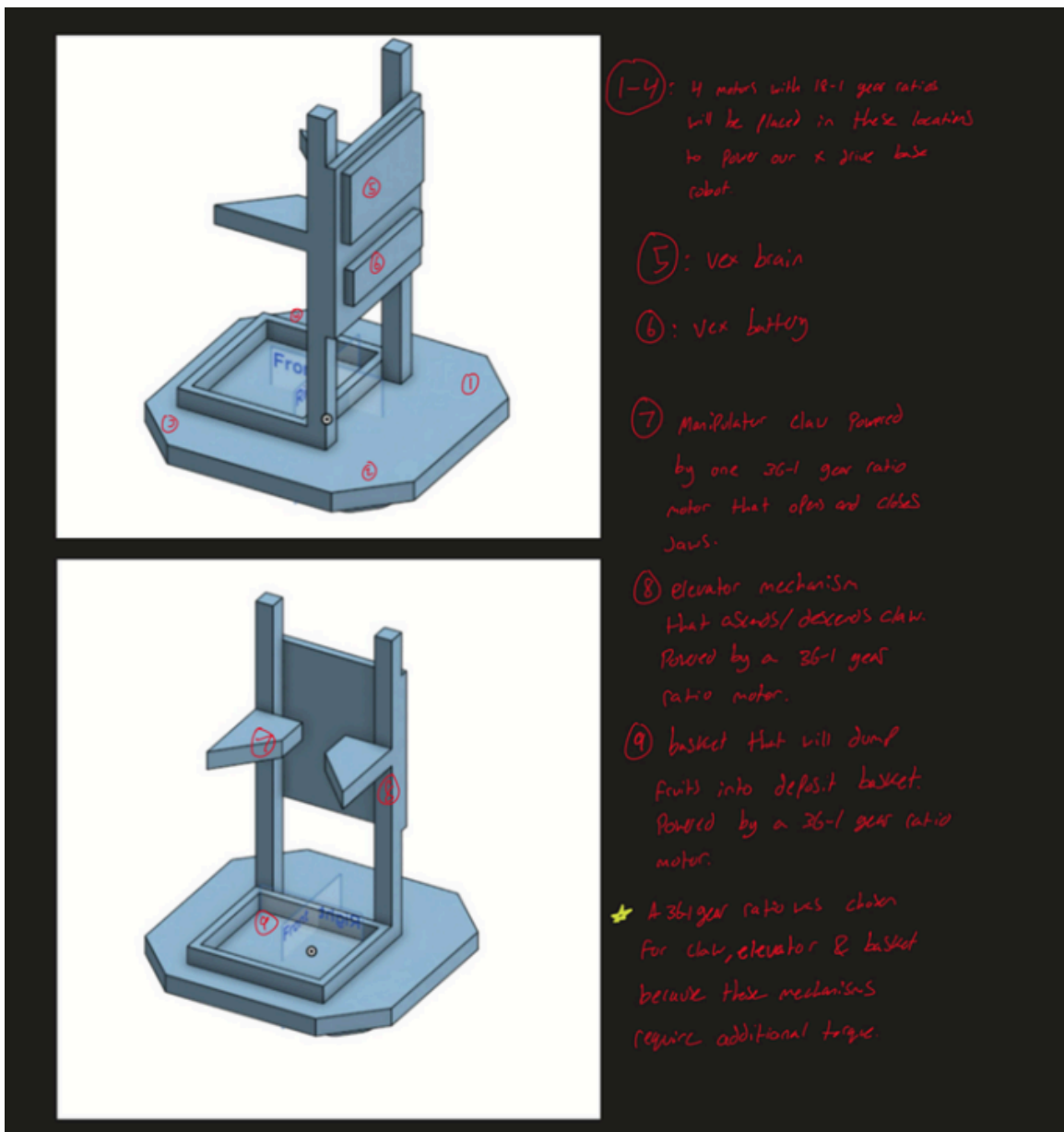
Requirement: Sensory Data

The system must be capable of avoiding obstacles using some navigational or sensory method.

Initial Concept

Here can be seen our notes regarding our initial robot concept. While the final project deviates from this mock up in places, the key components remained true until the final design. Early on, we knew we wanted a holonomic drivetrain, an elevator to adjust manipulator z-position, and some kind of dedicated fruit detachment mechanism mounted to the elevator.

Crayola CAD and Initial Planning

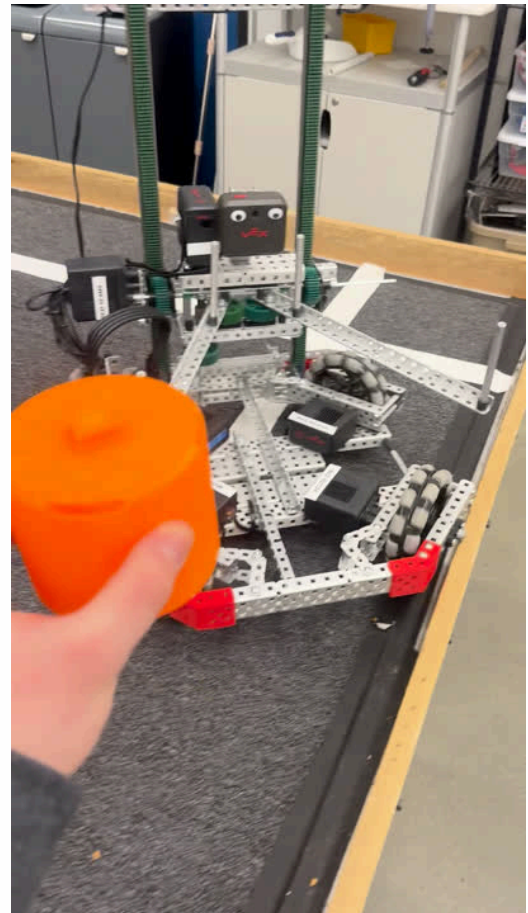


Design Philosophy

The overall design of this robot is the result of careful consideration for how it would ultimately be programmed. As students, we found that going into this project, we all needed the greatest improvement in the field of programming. Knowing that we had to deliver the final project within just a few short weeks, many of the decisions we made were to simplify the motion of the robot as much as possible, while still retaining the capability to traverse and harvest fruit from any field position. Our mechanisms were all designed with the goal of making our late stage programming as simple as possible by limiting the ranges of motion to the minimum number possible, adding physical sensors to limit motion, and designing of error possible during collection.

Holonomic Advantages

- While it does require more advanced programming than a tank or other similar drivetrain, a holonomic drivetrain simplifies overall programming by allowing the drivetrain to handle all x and y manipulator positioning, eliminating the need for a dedicated mechanism
- Despite requiring more complex math, having the drivetrain dynamically adjust to fruit position in real time greatly simplifies the challenge of positioning the robot in a position to reliably harvest the fruit every time



[^ VIDEO LINK ^](#)

Holonomic Drivetrain

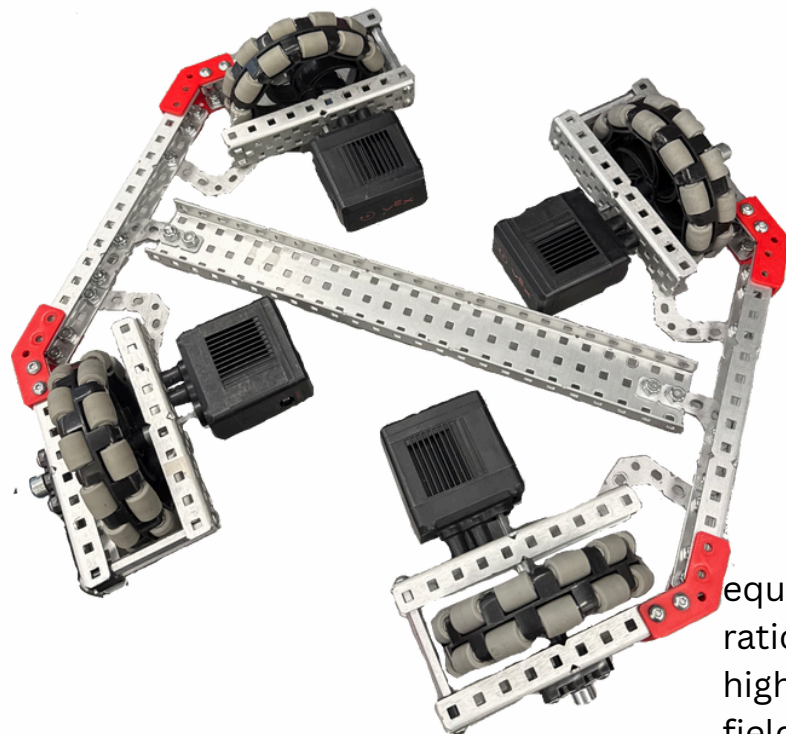
As previously noted, our group decided on a holonomic drivetrain early on as a goal for the project. A holonomic drivetrain was beneficial for two main reasons, the first being our goal to demonstrate proper robot code and develop our ability to do so, and the second being the advantages of enhanced maneuverability, despite having more mechanical simplicity. One of our other goals was to demonstrate an ability to work within design constraints. To fully accomplish this, we knew that we needed to create a robot which fit within the project's given size constraint. A holonomic drivetrain saves space in that the drivetrain is able to double as the manipulator's x and y adjustment, reducing the overall required number of mechanical components and allowing us to better meet our dimensional goals.

3D-Printed Brackets

- While the robot itself was mainly built from standard VEX hardware, few VEX standard components exist to achieve the required angles for an “X” style drivetrain
- To fix this, our team employed use of 3D-modeling and 3D-printing to implement a custom solution
- Other class members adopted and used this solution for their own projects

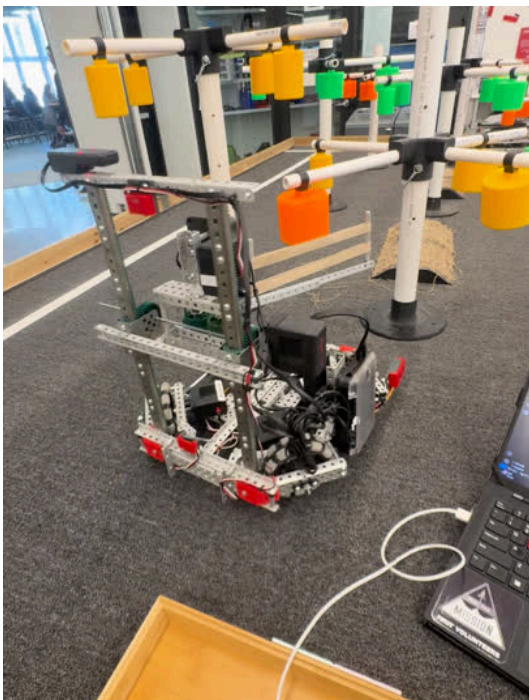
High-Torque Motors

During testing, the robot drivetrain was initially equipped with a set of motors with an internal gear ratio of 18:1. While these motors were capable of high speeds, the robot appeared to “slip” on the field, resulting in inaccurate movements. Through testing, it was found that by doubling the gear ratio to 36:1, the slip was eliminated, suggesting the robot had been lacking the torque needed for even motor acceleration.



Manipulator

The main manipulator mechanism of the robot is comprised of two key sub-systems: the Elevator and the Claw. The Elevator adjusts the Claw's Z-Height for adapting to different tree heights, while the Claw grips the fruit. To remove the fruit from the tree, the robot backs up while the claw is still clamped down and then drops the fruit into a storage basket located at the base of the robot.



[^ VIDEO LINK ^](#)

Elevator

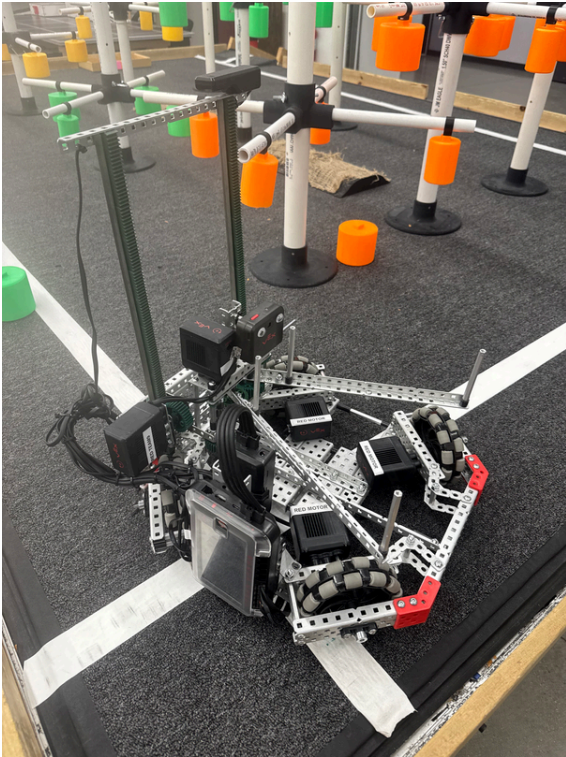
- Dual rail rack and pinion for adjustment
 - Powered by a single 11W motor with a 36:1 gear reduction
- Features two limit switches for autonomous zeroing and state switches

Claw

- Active collection
 - Powered by a single 11W motor with a 36:1 gear reduction
- Uses four elastic bands to increase friction between mechanism and fruit
- Intentionally designed to have a large margin of error during fruit alignment



Electronics / Sensor Array



[^ VIDEO LINK ^](#)

As per the class requirements, this robot utilizes the standard VEX V5 competition electronic hardware, with the exception of the camera. The camera itself is considered outdated VEX hardware, however it utilizes an open source and well documented vision system known better as the “Pixy-2” development board. All components were programmed using the Microsoft Visual Studio coding environment.

Pixy-2 Camera

- Open-source
 - Python , C & C++ programmable
- 60 FPS refresh rate
- Color-based filtering algorithm (CCC)
 - CCC stands for “Color Connected Components”
 - Algorithm calculates hue of each pixel
 - User maps hues to objects for recognition
 - Using hue over color makes the system more robust (hue changes less in alternate lighting)
- Memory capacity of 7 different signatures
 - Signatures can be individual colors or a pattern



[^ VIDEO LINK ^](#)

Programming - State Structure

Structure Choice

For this project, our team chose to code the robot using a State Machine structure for two main reasons.

The first reason was that the problem which the system solves can best be broken down into states. One way to think about programming this robot is to imagine it as an advanced vending machine. It must locate the desired item, collect the item, bring it to the deposit zone, and deposit the item. These are repeatable steps which will all be executed in the same order (with minor exceptions) every time the system is used, making a state based system an obvious choice.

The other reason was for troubleshooting simplicity. As previously stated, our group had the most room to grow in the field of programming, however we had limited time to implement the project. Not wanting to dedicate too much time to troubleshooting issues, we looked for a structure that would allow for easy troubleshooting. A state based system is extremely easy to troubleshoot because each state can be altered individually, meaning issues can be easily isolated and flaws with an individual state will typically not break the whole system.

Checkers & Handlers

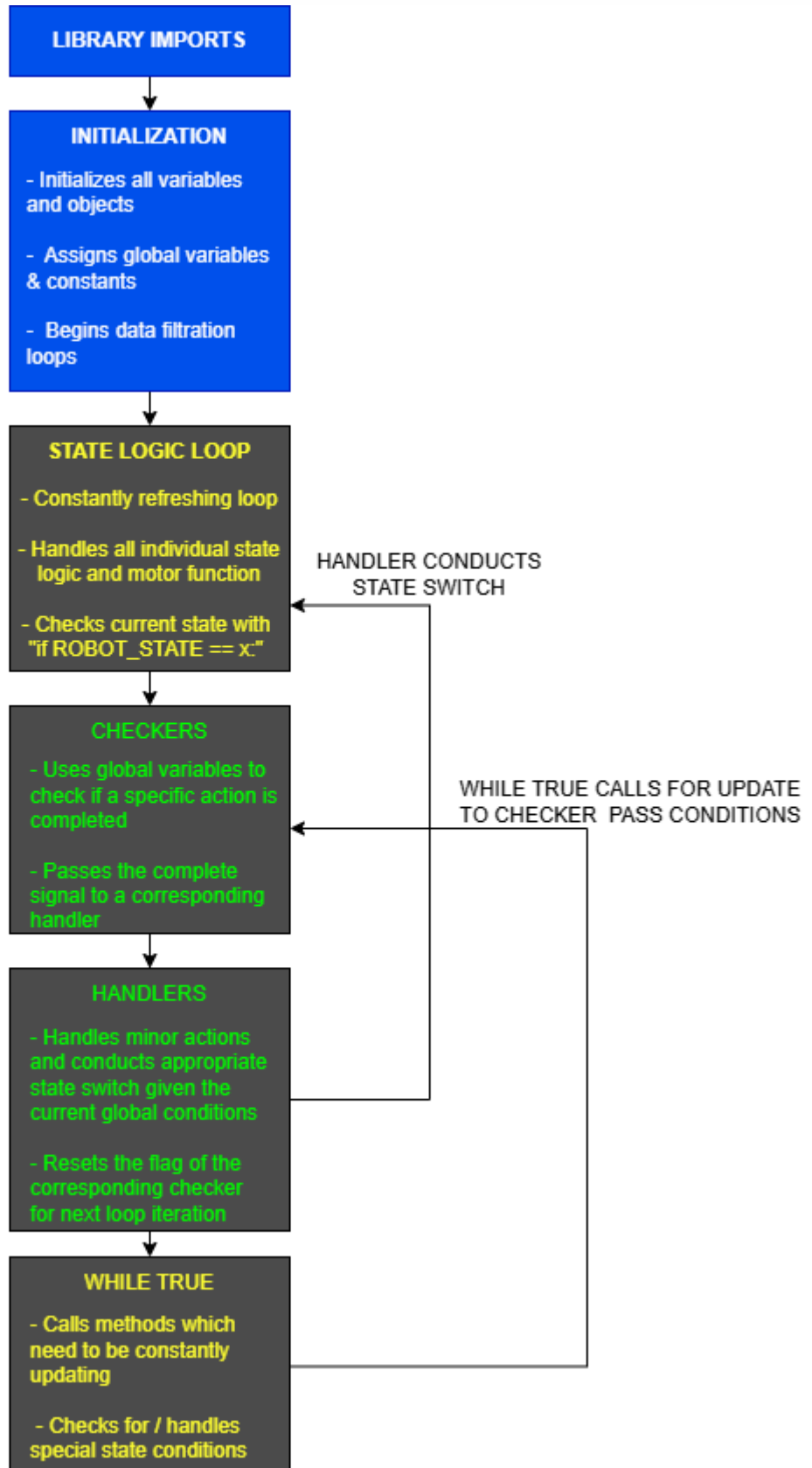
- Checkers check to see if the current action is completed by comparing end conditions to the current global conditions
- If they match, the checker raises a flag to call the corresponding handler
- The handler appropriately switches the state, resets local conditions, and lowers the flag
- The checker handler system is constantly running and refreshing

Sensor Data Filtration & Proportional Control

Certain sensor data, such as the line follow and ultrasonic sensor, are prone to “noise”, or brief inaccurate data outliers. For more accurate robot control, dedicated constant loops handle the filtering of said data. They function by running the filtration algorithm on the raw sensor data, and then replace all references to the sensor in the code with the filtered version of the data.

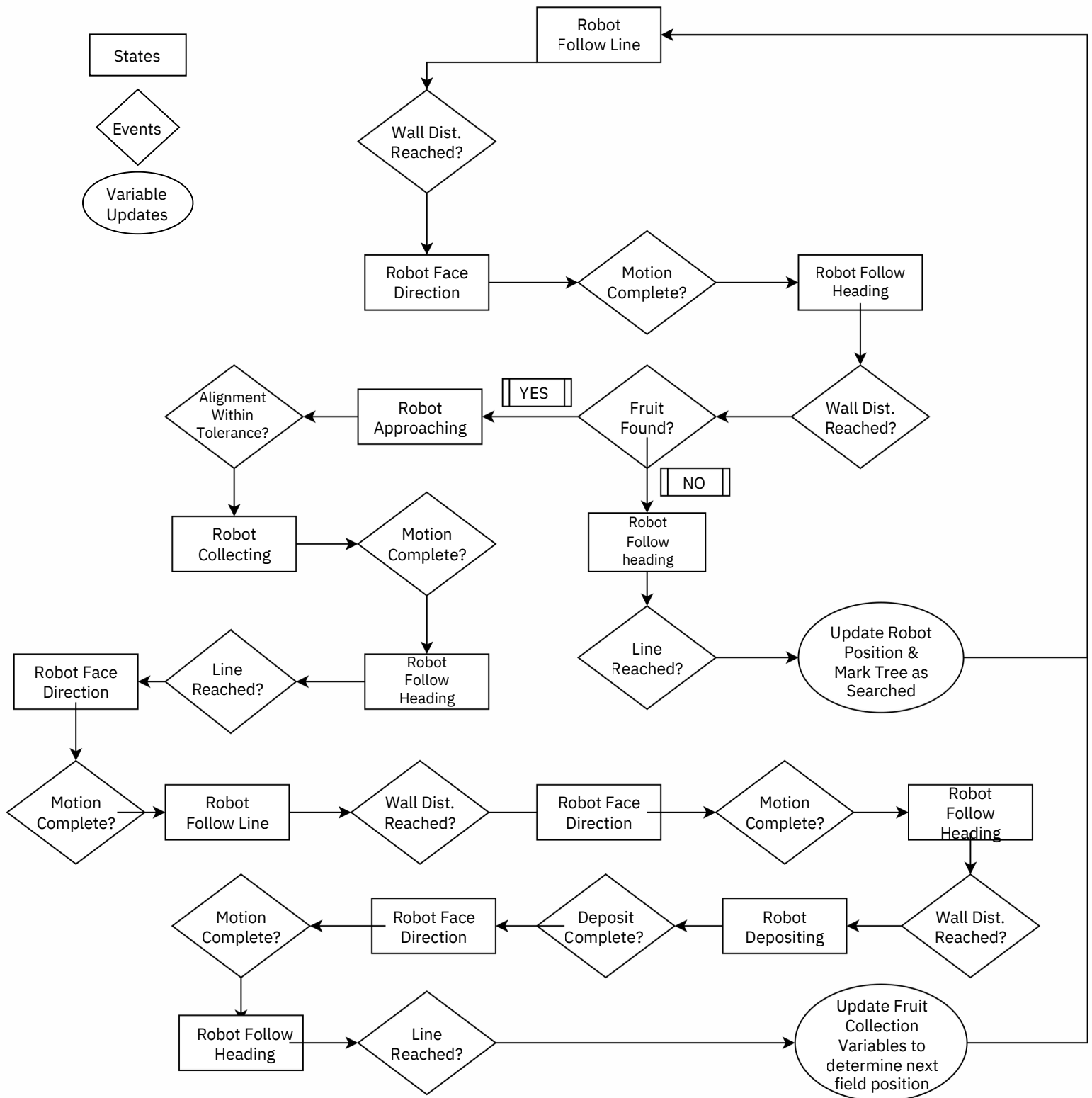
Programming - State Structure

State Machine Code Structure



Programming - State Structure

State Transition Diagram



Programming - Navigational Methods

Field-Oriented Control (FOC)

While requiring more advanced techniques initially, FOC greatly simplifies the amount of programming needed to execute proper navigation. Instead of maneuvering relatively, meaning directions like “left” and “right” change depending on robot orientation, the robot sets a permanent “north” direction upon initialization, and calculates directional movements based on the desired direction & orientation and the starting command.

For a task involving objects with fixed positions, this becomes especially useful, and it allows the robot to be programmed with commands which don't need to consider the current orientation of the robot. Like following a compass, all movements are based on the “North” heading the robot sets on initialization.

To **rotate** 45° about the origin, we apply the matrix

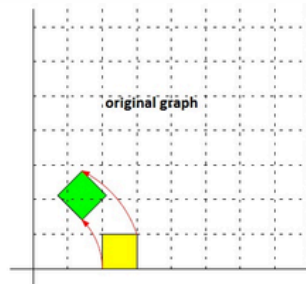
$$\begin{pmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{pmatrix} = \frac{\sqrt{2}}{2} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

Note: $\frac{\sqrt{2}}{2} = \cos 45^\circ = \sin 45^\circ$, so this is the same as

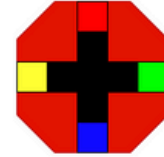
$$\begin{pmatrix} \cos 45^\circ & -\sin 45^\circ \\ \sin 45^\circ & \cos 45^\circ \end{pmatrix}$$

Counter Clockwise $\begin{pmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{pmatrix}$

Clockwise $\begin{pmatrix} \cos \phi & \sin \phi \\ -\sin \phi & \cos \phi \end{pmatrix}$

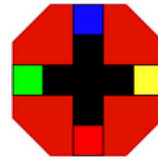


INITIALIZED ORIENTATION

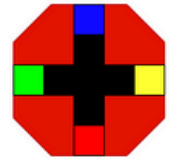


Note how RED is the “north” or “forward” heading

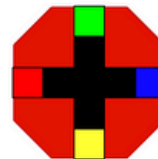
ORIENTATION BEFORE LEFT TURN COMMAND



ORIENTATION BEFORE LEFT TURN COMMAND

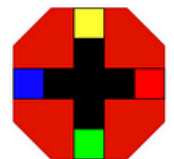


ORIENTATION AFTER LEFT TURN COMMAND (FOC)



Note how the robot has turned to face the left relative to the initialized orientation

ORIENTATION AFTER LEFT TURN COMMAND (NON-FOC)

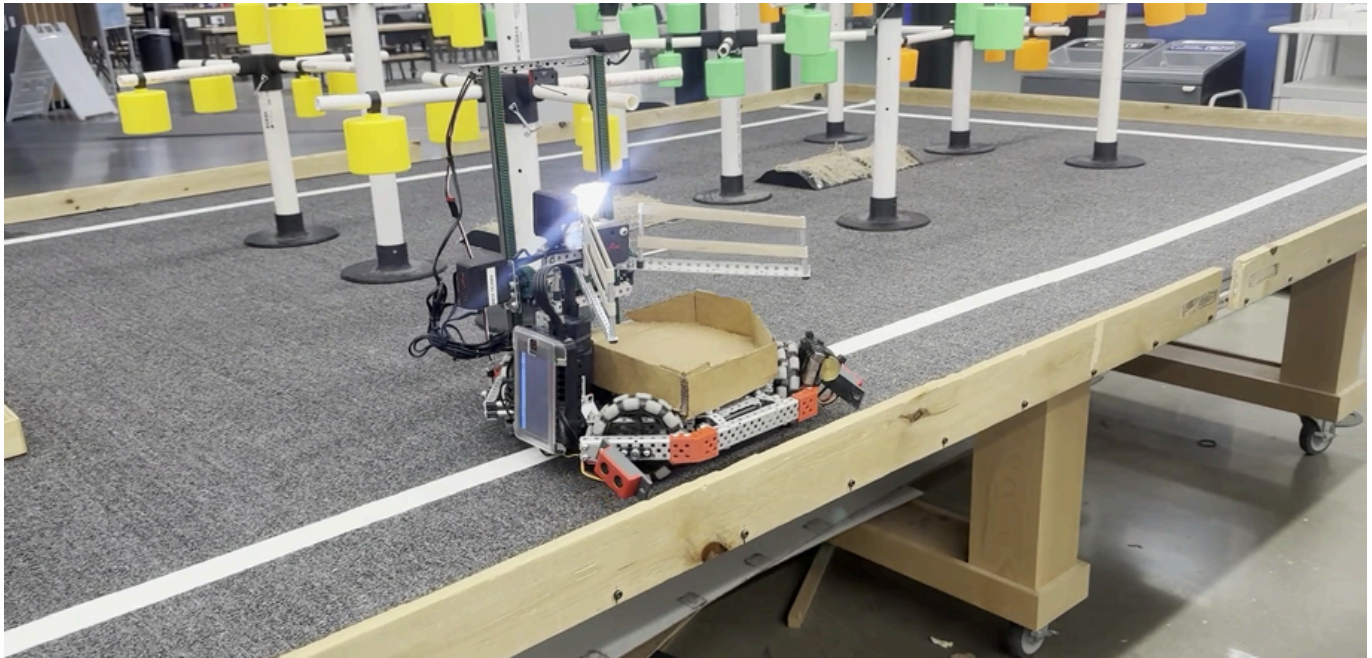


Note how the robot is now facing left relative to its previous orientation

The Rotation Matrix

To properly calculate robot orientation with holonomic control, a linear algebra based algorithm called the “Rotation Matrix” is applied to all drivetrain motor speed vectors. There is a parameter for forward movement, backward movement, and rotation applied to each motor. Different actions have different vector formulas using the Rotation Matrix algorithm.

Programming - Navigational Methods



Node Positioning System

[^ VIDEO LINK ^](#)

To navigate to different positions on the field to complete actions, the robot uses a node positioning system that assigns all spots where the robot can collect from a coordinate node. The coordinates are based off of the ultrasonic readings on the side of the robot. The X position is the position between the left and right side field walls, and the Y position is the distance from the front and back field walls. Using FOC control methods, the robot always knows which walls it's sensors are facing.

By combining these sensor readings with the FOC control methods of the robot, the robot can find its position at all times using only two ultrasonic sensors, minimizing the amount of sensor hardware needed for positional estimation.

This system also allows the robot to dynamically adapt with less coding complexity. As seen in the video, the robot misses the yellow fruit, but continues on to the next node it has been instructed to search. Without coordinate nodes, this would be harder to implement.

Github Link

[Click me!](#)